University of Engineering and Technology

New Campus, Lahore

Institute of AI, ML & Cyber Security

TECHNICAL REPORT

University Transport Management and Billing System

Submitted By

Name	Registration No.
Zuha Sheikh	2025-AI-566
Narmeen Abbas	2025-AI-554

Submitted To: Miss Namra Sheikh

1. Introduction

This technical report presents the complete design and implementation plan of a University Transport Management and Billing System, developed as a semester project for the course AI-134: Object Oriented Programming (OOP). The project demonstrates practical application of core OOP principles including inheritance, polymorphism, abstract classes, templates, file handling, and dynamic memory management.

Universities provide transport services to students and staff through buses and vans on fixed predefined routes. The manual management of such systems leads to numerous inefficiencies including overbooking, billing errors, and data loss. This project proposes a console-based, menu-driven C++ solution to automate the entire transport lifecycle.

2. Problem Statement

In many institutions, transport registration, route allocation, and fee management are handled manually, leading to the following critical issues:

- No centralized record of students using transport services
- Overbooking beyond vehicle seating capacity
- Manual fee calculation errors and late-payment inconsistencies
- No proper tracking of routes and assigned vehicles
- Absence of revenue and usage reports for administration
- Data is not stored persistently; lost after every session

This project designs and implements a console-based, menu-driven University Transport Management and Billing System using C++ and OOP principles to resolve all the above issues efficiently.

3. Course Learning Outcome (CLO-3)

Design and develop class hierarchies and object relationships including composition, inheritance, polymorphism, and template classes.

Bloom's Taxonomy Level: Analysis / Evaluation

4. CCP Attributes

A2 – Depth of Analysis Required

The problem has no single or obvious solution. Students must analyze real-world university transport management processes and convert them into a structured object-oriented system with clearly defined classes, responsibilities, and relationships.

A3 – Depth of Knowledge Required

The project requires strong understanding of the following OOP concepts:

- Inheritance and polymorphism
- Abstract classes and virtual functions
- Templates
- Dynamic memory management (new / delete)
- Operator overloading
- File handling for persistent storage

A8 – Interdependence

The system includes multiple interdependent modules: user management, transport management, route management, billing, and reporting. Any change in one module must correctly propagate to all related modules.

5. System Capabilities

Module	Features
Users	Admin (vehicles, routes, approvals, reports), Student (apply & track transport), Driver optional
Transport Mgmt	Add / Edit / Remove vehicles (Bus, Van), view available vehicles
Route Mgmt	Add routes (start, end, distance), assign vehicles, view details
Registration	Apply, Approve/Reject, Cancel, enforce capacity limits
Billing	Generate invoices, apply late fines, produce revenue & route-wise reports
Data Storage	Load from files at startup, save on exit, dynamic memory throughout

6. Object-Oriented Design & Class Hierarchy

The system is structured around the following class hierarchy implementing abstract base classes, derived classes, and supporting utility classes:

Class	Type	Responsibility
User	Abstract Base Class	Base for all users; pure virtual login()
Student	Derived: User	Applies for transport, views registration
Admin	Derived: User	Manages entire system, approves requests
Driver	Optional Derived: User	Assigned to specific vehicles
Vehicle	Abstract Class	Base for transport; pure virtual getType()
Bus	Derived: Vehicle	Large-capacity vehicle on fixed routes
Van	Derived: Vehicle	Smaller capacity, flexible assignment
Route	Concrete Class	Stores start, end, distance, vehicle list
TransportPass	Concrete Class	Links Student to Route
Bill / Invoice	Concrete Class	Manages monthly fee + late fines
Report	Utility Class	Generates revenue & usage reports

7. C++ Code Implementation

7.1 Abstract Base Class: User

The User class serves as the abstract base for all actors in the system. It defines a pure virtual login() method enforcing implementation in every derived class:

```
// User.h – Abstract Base Class
class User {
protected:
    string name, email, password;
    int userID;
public:
    User(int id, string n, string e, string p)
        : userID(id), name(n), email(e), password(p) {}
```

```
    virtual ~User() {}
    virtual void login() = 0;          // Pure virtual
    virtual void displayInfo() const = 0;
    string getName() const { return name; }
    int getID()   const { return userID; }
};
```

7.2 Derived Class: Student

Student inherits from User and adds transport-specific functionality including registration status and route tracking:

```
// Student.h – Derived from User
class Student : public User {
private:
    string regNo;
    TransportPass* pass;   // Association
public:
    Student(int id, string n, string e,
            string p, string reg)
        : User(id, n, e, p), regNo(reg), pass(nullptr) {}
    ~Student() { delete pass; }
    void login() override;
    void displayInfo() const override;
    void applyForTransport(Route* r);
    void cancelRegistration();
    TransportPass* getPass() { return pass; }
};
```

7.3 Abstract Class: Vehicle

Vehicle is an abstract class with a pure virtual getType() function. Bus and Van are concrete derived classes:

```
// Vehicle.h – Abstract Class
class Vehicle {
protected:
    int vehicleID, capacity, seatsAvailable;
    string plateNumber;
public:
    Vehicle(int id, int cap, string plate)
        : vehicleID(id), capacity(cap),
          seatsAvailable(cap), plateNumber(plate) {}
    virtual ~Vehicle() {}
    virtual string getType() const = 0;  // Pure virtual
    virtual void displayInfo() const = 0;
    bool hasSeats() const { return seatsAvailable > 0; }
    void bookSeat()   { if (hasSeats()) seatsAvailable--; }
    void cancelSeat() { seatsAvailable++; }
};

// Bus and Van derived classes
class Bus : public Vehicle {
public:
    Bus(int id, string plate)
        : Vehicle(id, 50, plate) {}
    string getType() const override { return "Bus"; }
    void displayInfo() const override;
};

class Van : public Vehicle {
public:
    Van(int id, string plate)
        : Vehicle(id, 14, plate) {}
    string getType() const override { return "Van"; }
```

```
    void displayInfo() const override;
};
```

7.4 Template Class: DataStore<T>

A generic template class is used to store any type of object (vehicles, routes, users) with dynamic memory management:

```
// DataStore.h – Template Class
template <typename T>
class DataStore {
private:
    T** items;
    int count, capacity;
public:
    DataStore(int cap = 100)
        : count(0), capacity(cap) {
        items = new T*[capacity];
    }
    ~DataStore() {
        for (int i = 0; i < count; i++) delete items[i];
        delete[] items;
    }
    void add(T* item) {
        if (count < capacity) items[count++] = item;
    }
    T* get(int index) { return items[index]; }
    int getCount()   { return count; }
};
```

7.5 Operator Overloading in Bill Class

The Bill class overloads the += operator to allow adding late fines directly to the existing fee amount:

```
// Bill.h – Operator Overloading
class Bill {
private:
    int billID;
    float monthlyFee, lateFine, totalAmount;
    bool isPaid;
public:
    Bill(int id, float fee)
        : billID(id), monthlyFee(fee),
          lateFine(0), isPaid(false) {
        totalAmount = fee;
    }
    // Operator overload: bill += fine
    Bill& operator+=(float fine) {
        lateFine    += fine;
        totalAmount += fine;
        return *this;
    }
    void markPaid()  { isPaid = true; }
    float getTotal() { return totalAmount; }
    void printInvoice() const;
};
```

7.6 File Handling – Save & Load Data

All data (users, vehicles, routes, registrations) is persisted to text files. Data is loaded at startup and saved before exit:

```
// FileHandler.cpp – Save & Load
#include <fstream>
#include <sstream>
using namespace std;

void FileHandler::saveVehicles(DataStore<Vehicle>& store) {
    ofstream file("vehicles.dat");
    for (int i = 0; i < store.getCount(); i++) {
        Vehicle* v = store.get(i);
        file << v->getID() << ","
             << v->getType() << ","
             << v->getPlate() << "\n";
    }
    file.close();
}

void FileHandler::loadVehicles(DataStore<Vehicle>& store) {
    ifstream file("vehicles.dat");
    string line;
    while (getline(file, line)) {
        stringstream ss(line);
        string id, type, plate;
        getline(ss, id,    ',');
        getline(ss, type,  ',');
        getline(ss, plate, ',');
        if (type == "Bus")
            store.add(new Bus(stoi(id), plate));
        else
            store.add(new Van(stoi(id), plate));
    }
    file.close();
}
```

8. Object Relationships in Design

Relationship	Classes	Description
Association	Student – TransportPass	Student is linked to a transport pass
Aggregation	TransportManager – Vehicles / Routes	Manager holds but does not own them
Composition	TransportPass – Bill	Bill cannot exist without TransportPass

9. Menu-Driven Interface

The system provides role-based menus. Upon login, users are directed to their respective menu:

```
========== MAIN MENU ==========
  1. Register User
  2. Login
  3. Exit
================================

======== STUDENT MENU ==========
  1. View Available Routes
  2. Apply for Transport
  3. View My Registration
  4. Cancel Registration
  5. Logout
================================
```

```
========= ADMIN MENU ===========
  1. Add Vehicle
  2. Edit Vehicle
  3. Remove Vehicle
  4. Add Route
  5. Assign Vehicle to Route
  6. View Applications
  7. Approve / Reject Requests
  8. Generate Reports
  9. Save Data
 10. Logout
================================
```

10. Dynamic Memory & Data Management

- All objects dynamically allocated with new, deallocated with delete
- Destructor chain ensures no memory leaks across class hierarchy
- Template DataStore<T> manages all collections generically
- Files loaded at startup: users.dat, vehicles.dat, routes.dat, bills.dat
- All data saved before program exit via FileHandler::saveAll()

11. Assumptions

- Each vehicle has a fixed, predefined seating capacity
- Each student registers for only one transport route at a time
- Transport fees are calculated monthly
- Late payment fines apply automatically after due date
- Only approved registrations occupy vehicle seats
- Data stored in text or binary files for persistence

12. Deliverables & Assessment Rubric

Deliverable	Evaluation Criterion
Technical Report (PDF)	Problem understanding, design clarity, assumptions
UML Class Diagram	Correctness of class hierarchy and relationships
Source Code (.h and .cpp)	OOP concepts, dynamic memory, file handling
Program Screenshots	All modules working, clean output format

13. Conclusion

The University Transport Management and Billing System demonstrates practical mastery of Object-Oriented Programming in C++. By modeling real-world entities — users, vehicles, routes, registrations, and bills — as well-defined classes with clear relationships, this project bridges theoretical OOP knowledge with software engineering practice.

The use of abstract classes, template containers, operator overloading, and file persistence ensures the system is scalable, maintainable, and industry-relevant. Successful completion of this project validates CLO-3 achievement and prepares students for professional C++ software development.